



SIMULADOR DE INTERCAMBIO DE CROMOS — MUNDIAL FIFA 2026



Integrantes:

- Danny Caiza
- Melanie Peñafiel
- Eduardo Verdezoto
- Alexander Mena



OBJETIVO

Determinar el número promedio de fundas adicionales necesarias para completar el Álbum Mundial FIFA 2026 en poblaciones cerradas de distintos tamaños, analizando el efecto de los intercambios entre participantes.



DATOS DEL PROBLEMA

- 980 cromos en total.
- 7 cromos por funda.
- Costo de cada funda: \$1.20.
- Participantes pueden intercambiar duplicados.



REPRESENTACIÓN DE LA COLECCIÓN

Cada participante posee un arreglo de tamaño 980.

Ejemplo: [1,0,2,1,0,3,...]

- 0 → cromos faltante.
- 1 → cromo obtenido.
- 2 o más → cromos duplicados.

FUNCIONES PRINCIPALES



```
def get_missing(col):  
    return int(np.sum(col == 0))
```

Cuenta cromos faltantes

```
def get_dups(col):  
    return int(np.sum(np.maximum(col - 1, 0)))
```

Cuenta cromos Repetidos

Generación aleatoria de cromos

Función

```
def buy_packs(col, n):  
    idx = np.random.randint(0, TOTAL, n * PER_PACK)  
    np.add.at(col, idx, 1)
```

Proceso

- Comprar fundas.
- Cada funda contiene 7 cromos.
- Los cromos se generan aleatoriamente.
- Se actualiza la colección.



ALGORITMOS DE INTERCAMBIO



DIRECTO

```
def exchange_round(parts):  
    n = len(parts)  
    swaps = 0
```

- Recorre cada participante i.
- Busca los cromos que le faltan.
- Revisa si otro participante j tiene ese cromo duplicado.
- Si lo tiene:
- Se le quita una copia al participante j.
- Se le entrega al participante i.
- Se registra un intercambio (swaps += 1).
- Actualiza las listas de faltantes y duplicados.

```
for i in range(n):  
    if not missing_sets[i]:  
        continue  
  
    for sticker in list(missing_sets[i]):  
        for j in range(n):  
            if i == j or sticker not in dups_sets[j]:  
                continue  
  
            parts[j][sticker] -= 1  
            parts[i][sticker] += 1  
            swaps += 1  
            progress = True  
            missing_sets[i].remove(sticker)  
            if parts[j][sticker] <= 1:  
                dups_sets[j].discard(sticker)  
            break
```


INTERCAMBIOS TRIANGULARES

- Identificar tres participantes.
- Cada uno posee un duplicado útil para otro.
- Se realiza un intercambio circular.
- Los tres obtienen cromos faltantes.

```
for k in range(n):
    if k == i or k == j:
        continue

    candidates_j = missing_sets[j] & dups_sets[k]
    if not candidates_j:
        continue

    candidates_k = missing_sets[k] & dups_sets[i]
    if not candidates_k:
        continue

    sticker_ij = next(iter(candidates_i))
    sticker_jk = next(iter(candidates_j))
    sticker_ki = next(iter(candidates_k))

    parts[j][sticker_ij] -= 1
    parts[i][sticker_ij] += 1
    parts[k][sticker_jk] -= 1
    parts[j][sticker_jk] += 1
    parts[i][sticker_ki] -= 1
    parts[k][sticker_ki] += 1

    swaps += 3
```



ESTIMACIÓN DE FUNDAS ADICIONALES

```
def extra_packs_needed(missing):  
    if missing == 0:  
        return 0  
  
    return int(  
        np.ceil(  
            (TOTAL / PER_PACK)  
            * np.log(  
                TOTAL / (TOTAL - missing)  
            )  
        )  
    )
```

CALCULA CUÁNTAS
FUNDAS ADICIONALES
COMPRAR CON EL FIN DE
REDUCIR COMPRAS
INNECESARIAS.

$$N = \left\lceil \frac{980}{7} \ln \left(\frac{980}{980-m} \right) \right\rceil$$

SIMULACIÓN COMPLETA



Esta función simula el proceso completo para un grupo de participantes.

- Se crean los álbumes de todos los participantes.
- Cada participante compra fundas iniciales.
- Se realizan intercambios de cromos.
- Se calculan los cromos faltantes.
- Si aún faltan cromos, se compran fundas adicionales.
- El proceso se repite hasta completar los álbumes.

```
def simulate_one(n_part, max_rounds=None):
    parts = [np.zeros(TOTAL, dtype=np.int32) for _ in range(n_part)]
    for p in parts:
        buy_packs(p, INIT_PACKS)

    extra_total = 0
    rounds = []
    r = 0

    while True:
        r += 1
        swaps = exchange_round(parts)
        miss_before = [get_missing(p) for p in parts]
        extra = 0

        for p in parts:
            missing = get_missing(p)
            if missing == 0:
                continue

            packs_needed = extra_packs_needed(missing)
            buy_packs(p, packs_needed)
            extra += packs_needed
```


ANÁLISIS ESTADÍSTICO

Ejecución de varias simulaciones con el fin de obtener resultados más confiables.

- Una sola simulación puede verse afectada por el azar.

Resultados obtenidos

- Fundas extra promedio.
- Álbumes completados.
- Rondas promedio.
- Costo promedio por persona.

```
def simulate_many(n_part, reps, max_rounds, progress_cb=None):  
    results = []  
    for i in range(reps):  
        results.append(simulate_one(n_part, max_rounds))  
        if progress_cb:  
            progress_cb(i + 1, reps)  
  
    avg_extra = np.mean([r["extra"] for r in results])  
    avg_done = np.mean([r["done"] for r in results])  
    avg_rounds = np.mean([len(r["rounds"]) for r in results])  
    avg_cpp = (INIT_PACKS + avg_extra / max(n_part, 1)) * COST
```



CONCLUSIONES



- El intercambio de cromos entre participantes reduce la cantidad de cromos faltantes y disminuye la necesidad de comprar fundas adicionales para completar el álbum.
- La repetición de múltiples simulaciones permite obtener resultados más confiables sobre el comportamiento real del proceso de completar el álbum.
- A medida que aumenta el número de participantes, disminuye el costo promedio por persona debido al incremento de oportunidades de intercambio.



GRACIAS

